

STAT 390 Presentation 1

Sid Chalasani, Kai Neumark, Andrew Ross, Dustin Zhao, Eric Zhou

Problem Statement

Why Classifying C-MIL Matters

- Conjunctival melanocytic intraepithelial lesions (C-MIL) → tissue that consist of the melanocytes within the epithelial region of the conjunctiva
- Pre-invasive melanoma → potentially controlled with treatment
- Correct classification → patient management
 - Benign melanosis: no treatment
 - Low-grade C-MIL: observation
 - High-grade C-MIL: (aggressive) treatment
- Misclassification is harmful
 - Overtreatment → unnecessary side effects for more healthy patients
 - Undertreatment → development of invasive melanoma for more sick patients

Limitations of Expert-Only Classification

- Classification is performed by ocular pathologists using tissue-slice images
- Decisions rely on visual judgement of melanocyte distribution
- There is substantial disagreement among experts
 - Especially for low-grade C-MIL (most difficult category)
- In a published study by the stakeholder, expert agreement with the ground truth was ~39% for low-grade cases
- **Result:** diagnosis is subjective, inconsistent, not reproducible
- **Significance:** we need to build a really strong ML model to assist the experts

Formal Problem Statement

- Input: histopathology tissue slices of patients' conjunctiva
 - Multiple stains (H&E, Melan-A, SOX10 – all necessary)
 - Multiple patches per slice
 - Multiple slices per patient
- Relevant region: the epithelial layer of the conjunctiva
- Key signal: distribution of melanocytes across the epithelium (base = benign, middle = low-grade, entire-width = high-grade)
- Objective: develop a ML model to classify C-MIL cases
- Constraints:
 - Labels are at the patient level, not slice or patch level
 - Each patient consists of multiple slices, patches, and stains
 - Since the same slice cannot be stained multiple times, images across stains can only be matched approximately

Math

Data Pre-Processing

- We want to do MIL!
- Hierarchy: Case (Benign/High Risk) → Stain → Slice → Patches
- 60/20/20 Train, Test, Validation Split
- Max of 800 Patches per Slice
- Optional Cap on Slices per Stain

Model Training

- Patch feature extraction
 - DenseNet-121 (Pretrained, Frozen) $\rightarrow$ 1024 7x7 feature maps
 - AdaptiveAvgPool2d $\rightarrow$ 1024 2x2 feature maps
 - Flattened feature size = $4F \rightarrow x_p \in \mathbb{R}^{4096}$
- Linear projection (trainable)

$$z_p = W_{\text{proj}} x_p + b_{\text{proj}}, \quad W_{\text{proj}} \in \mathbb{R}^{512 \times 4096}$$

- Output patch embedding $\rightarrow z_p \in \mathbb{R}^{512}$

Attention Math

Patch-level attention:

$$a_p = v^\top \tanh(W_a z_p)$$

$$W_a \in \mathbb{R}^{128 \times 512}$$

$$\alpha_p = \frac{e^{a_p}}{\sum_{j=1}^P e^{a_j}}$$

$$v \in \mathbb{R}^{128}$$

$$s = \sum_{p=1}^P \alpha_p z_p \in \mathbb{R}^{512}$$

Same attention math at every level!

Patch $\rightarrow$ Slice $\rightarrow$ Stain $\rightarrow$ Case

Training & Validation

Final Classifier:

Linear layer $\rightarrow \mathbb{R}^{512} \rightarrow \mathbb{R}^2 \quad \mathbf{o} = \mathbf{W}_{\text{cls}} \mathbf{z}_{\text{case}} + \mathbf{b}_{\text{cls}}$

Loss Function:

Weighted cross-entropy loss $\rightarrow \mathcal{L} = -w_y \log P(y \mid \text{case})$

Class weights:

Benign (0): $w_0 = 2.5$, High-grade (1): $w_1 = 1.0$

Optimization:

Adam, LR = 3×10^{-4} , Weight Decay = 2×10^{-4} , LR Scheduler, Early Stopping

Training, Forward and Back Pass

Forward Pass:

$$\mathbf{o} = f_{\theta}(\text{case}) \in \mathbb{R}^2 \quad P(y = k \mid \text{case}) = \frac{e^{o_k}}{\sum_{j=1}^2 e^{o_j}}$$

Loss:

$$w_0 = 2.5 \quad (\text{benign}), \quad w_1 = 1.0 \quad (\text{high-grade})$$

$$\mathcal{L} = -w_y \log P(y \mid \text{case})$$

Backpropagation: Through final classifier, attention layers, and patch projector

$$\nabla_{\theta} \mathcal{L} = \frac{\partial \mathcal{L}}{\partial \mathbf{o}} \frac{\partial \mathbf{o}}{\partial \theta}$$

Adam Optimizer Update: $\theta \leftarrow \theta - \eta \cdot \text{Adam}(\nabla_{\theta} \mathcal{L})$

Where η is learning rate. Adam uses adaptive scaling and momentum for η .

Weight Decay:

$$\mathcal{L}_{\text{total}} = \mathcal{L} + \lambda \|\theta\|_2^2$$

We learn...

1. Patch Projection $W_{\text{proj}}, b_{\text{proj}} \quad (4096 \rightarrow 512)$
2. Attention Params $W \in \mathbb{R}^{128 \times 512}, \quad v \in \mathbb{R}^{128}$
3. Final Classifier $W_{\text{cls}} \in \mathbb{R}^{2 \times 512}$

DenseNet is frozen and not learned!

Model Prediction

1. Compute attention embeddings

2. Produce logits $\text{logits} = (l_0, l_1)$

3. Convert to probabilities $P(y = k) = \frac{e^{l_k}}{\sum_j e^{l_j}}$

4. Predicted $\hat{y} = \arg \max_k P(y = k)$

Attention Hierarchy $\rightarrow \alpha_p^{\text{case}} = \alpha_s^{\text{stain}} \times \alpha_j^{\text{slice}} \times \alpha_p^{\text{patch}}$

Outputs saved:

Per-case model predictions and probabilities, confusion matrix, data splits

Programming

Understanding the Code

- Main.py is the core training script for this MIL model

Workflow:

1. Load labels and patches
2. Split by case (stratified)
3. Create datasets
4. Build model
5. Train with validation
6. Evaluate on test set
7. Save & visualize results

Python Files

main.py - Main pipeline file

config.py - Defines all hyperparams, paths, and settings

data_utils.py - Contains all data preprocessing and organization utilities

models.py - Defines the neural network architecture for the MIL model

dataset.py - Implements pytorch dataset class that loads image patches with configs

trainer.py - Implements training, validation, and evaluation logic for the model

utils.py - General helper functions for reproducibility, debugging, monitoring, data analysis

attention_analysis.py - Implements attention analysis and visualization to understand what the model is focussing on for the predictions

Execution Flow - main.py

1. Data Preparation - data_utils.py
2. Dataset & Data Loader - dataset.py
3. Model Architecture - models.py
4. Training Loop - trainer.py
5. Evaluation - trainer.py
6. Attention Analysis - attention_analysis.py

Data Preparation - dataset.py & data_utils.py

a. Load & Parse data

- i. Loads CSV Paths
- ii. Loads & groups patch data by case_id, slice_id using regex
- iii. Maps slices to binary labels

b. Stratified Splitting

- i. Split by CASE to prevent data leakage, uses sklearn.train_test_split with stratification

c. Build Case Dictionaries

- i. Reorganizes slice-level lists into the nested hierarchical structure required by the MIL model

d. Data Integrity Checks

- i. Verifies for no case/path overlap, reports empty cases and missing stains
- ii. Saves case IDs for reproducibility

Model Architecture - models.py

- Uses sequential stain processing
 - Processes stains one at a time in a loop
- Stochastic Patch Sampling as Data Augmentation
 - Makes sure model sees different patch combinations across training iterations
- Lazy Loading with batch size constraint
 - Loads images on demand
- Frozen feature extractor with gradient isolation
 - Reduces usable params to save GPU memory
- Hierarchical tensor reduction using attention pooling
 - Model progressively reduces dimensionality through attention

Training Infrastructure - trainer.py

Execution Engine, receives data loaders and executes training

- **Weighted Cross-Entropy Loss**
 - Penalizes benign class errors by 2.5x
- **Dimension Manipulation for Single-Case Processing**
 - Reduces dimension to a dimensions expected by CrossEntropyLoss
- **Early Stopping with Configurable Patience and Minimum Epochs**
 - Implements two conditions to prevent premature stopping
- **ReduceLROnPlateau as Scheduler with Validation Loss Monitoring**
 - If validation loss does not decrease for 3 epochs, learning rate is reduced
- **Checkpoint State Dictionary with Full Training History**
 - Stores data from training

Analysis Tools - attention_analysis.py

Enables model interpretability, runs after training and evaluation is complete, triggered by `--analyze_attention` flag

- Effective Patch Attention using Three Level Weight Multiplication
 - Shows global patch importance across entire case
- Per Case Histogram Generation
 - Bins weights then normalizes counts by total patches
- Top Patch Selection
 - Helps understand which tissue types and regions the model considers most important
- Slice Attention Rank Plots
 - Plots slice weights to identify attention concentration
- Dual Patch Visualization
 - Generates top/bottom N patches within each slice and top/bottom N patches across the entire case

Code Efficiency & Redundancy - main.py

Code is solid overall but a few minor improvements:

- `prepare_data()`:
 - Currently: Creating train, test, and validation slices goes through every element of `slice_to_class` three times
 - A more efficient way to implement would be to iterate through `slice_to_class.keys()` once and append to `train_slices`, `val_slices`, or `test_slices` based on the `case_id` of each
- `create_data_loaders()`:
 - Currently: Dataloaders kill worker processes after each epoch
 - Add `persistent_workers=True` so that workers can remain for each epoch, reducing the time it takes to transition between epochs

Consulting / Analysis

Current Methodology: High-Level Overview

1. Objective: Classify conjunctival melanocytic intraepithelial lesions (C-MIL) as...

- Benign
- High-grade (cancerous)

2. Data:

- Microscopic tissue images from multiple hospitals
- Labels available only at the case (patient) level

3. Approach:

- Pre Trained CNNs for feature extraction
- Multiple Instance Learning (MIL)
- Attention-based pooling to aggregate patch information

Key idea: Learn patch-level features → aggregate them → predict case-level severity

Model Structure Summary

Biological hierarchy:

- Case (patient)
 - Different stains (H&E, Melan-A, Sox10)
 - Multiple slices per stain
 - Many patches per slice

1. Patch-level processing:

- Each patch passed through a pretrained CNN
- Output: fixed-length embedding (~512 dimensions)

2. Aggregation:

- Attention-based MIL pooling
- Learns weights indicating “important” patches

3. Prediction:

- Weighted bag-level embedding
- Linear classifier outputs probability of high-grade C-MIL

Ad-Hoc & Underdeveloped Aspects

1. No explicit spatial or geometric modeling within slices

- Patches treated as a set within each slice
- No positional encoding, graph structure, spatial bias in the pipeline

2. Attention as weighting, not relational learning

- Since attention is applied after patch features are extracted, it only reweights existing representations rather than learning interactions
- The model can't capture co-occurrence or contextual relationships

3. Pre-trained backbone not domain-specific

- DenseNet121 trained on natural images, not histopathology
- Limits feature expressiveness for cellular morphology

Improvements on the Current Method

1. Use domain-specific pretrained models

- Replace DenseNet121 with KimiaNet, UNI, or other histopathology-trained CNNs
- Improve sensitivity to tissue-specific texture patterns

2. Make attention more expressive

- Extend attention beyond scalar weighting (gated, multi-head, interaction-aware)
- Allow the model to recognize contextual relationships across regions

3. Strengthen regularization and training stability

- Explicitly regularize attention weights and intermediate embeddings to reduce over-confident focus on spurious regions
- Improve generalization and robustness without changing the core model structure

